claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_b05b902c-a7c\agent-a2b90993b5d5d9ebf.jsonl`
- SHA-256: `02b369be78d1bbbd3579a9ab473eb240d52ad17b60cb7eea4592c864224afa9f`
- Source modified: `2026-06-19T16:32:50+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b05b902c-a7c`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`

Transcript

1. user (2026-06-19T16:29:39.645Z)

Adversarially VERIFY this prominent-court-gate result for the multicourt clip Badminton_BXH_2. Independently re-run the measurement (recipe below) and check: (1) the polygon is RECALL-SAFE (pct_golden_in_polygon >= 0.85) and excludes a genuine adjacent court (not just drawn to match golden); (2) the precision/F1 gain is REAL and the dropped rallies are genuinely FALSE POSITIVES (matched no golden), not real rallies; (3) recall did NOT drop materially; (4) no number is inflated. If the reported result is wrong or inflated, return the CORRECTED row; if recall was silently lost, fix delta_recall + read. Output the verified/corrected row.

REPORTED ROW:

```
{
 "name": "Badminton_BXH_2",
 "n_gt": 44,
 "pct_golden_in_polygon": 1,
 "polygon_summary": "Quad [(0.0,0.08),(0.68,0.08),(0.68,0.68),(0.0,0.68)] in normalized 0-1 coords. Derived from the 2D distribution of visible shuttle points (FW=1920, FH=1080): all shuttle mass and all 44 golden-rally medians (x 0.098-0.638, y 0.246-0.429) live in the left/center foreground court; the right (x>0.68) and bottom (y>0.68) of the frame carry essentially no shuttle. Contains 100% (44/44) of golden medians. There is NO spatially-separated secondary/adjacent-court cluster to exclude in this clip, so this recall-safe polygon is effectively a near-full-court box = a no-op.",
 "base_n_pred": 78,
 "base_precision": 0.346154,
 "base_recall": 0.613636,
 "base_f1": 0.442623,
 "base_tolf1": 0.327869,
 "gated_n_pred": 78,
 "gated_precision": 0.346154,
 "gated_recall": 0.613636,
 "gated_f1": 0.442623,
 "gated_tolf1": 0.327869,
 "n_dropped": 0,
 "n_dropped_were_fp": 0,
 "delta_precision": 0,
 "delta_recall": 0,
 "delta_f1": 0,
 "read": "HONEST NULL: recall-safe prominent-court polygon is a no-op (drops 0/78, all metrics identical). The clip over-fires hard (78 pred vs 44 gt, P=0.346) but the FPs are spatially CO-LOCATED with TPs in the prominent court (FP median x 0.060-0.497 == TP x 0.098-0.484; ~42% of all visible points are a static tracker artifact parked at court-center 0.467,0.347). No separable adjacent-court cluster exists, so a spatial court gate cannot cut these FPs; tightening the polygon to force drops immediately kills recall (drops real rallies, not a background court). No win here for this gate."
}
```

```
RECIPE:
You are measuring the prominent-court shuttle gate's quality win on ONE multicourt badminton
golden clip, against human golden labels. Work in the repo root; run Python via Bash. Be
numerically exact ? do NOT estimate.

DATA
- Golden rallies: json.load(open("output/Badminton_BXH_2_golden_features.json"))["gts"] -> list
of [start,end] seconds.
- Manifest output/human_lovo_manifest.json: the entry with name==Badminton_BXH_2 gives
trajectory (path), fps, frame_width (=1920 for these; so frame_height=1080).

STEP 1 baseline (ungated) local rallies:
  from backend.eval.rally_seg_eval import windows_for
  from backend.eval.windowing import SERVED
  video={"name":NAME,"trajectory":TRAJ,"fps":FPS,"frame_width":FW,"gts":GTS}
  baseline = windows_for(video, SERVED)

STEP 2 derive a normalized (0-1) prominent-court polygon, RECALL-SAFE:
  from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
  points = TrackNetRunner.parse_trajectory_csv(TRAJ)   # each has .visible, .x, .y, .frame
  fh = 1080.0
  Inspect the 2D distribution of visible (x/FW, y/FH). Multicourt clips can have multiple
shuttle clusters (courts); the PROMINENT court = the dominant/foreground cluster (most shuttle
mass; foreground tends lower/larger in frame). Draw a polygon (quad or bounding box in
normalized coords) around the prominent cluster, EXCLUDING any clearly-separated secondary
(adjacent) cluster.
  VALIDATE recall-safety: for each golden rally compute its shuttle MEDIAN (x_norm,y_norm) over
frames in [s*FPS, e*FPS]; your polygon MUST contain >= 0.85 of golden-rally medians
(point_in_polygon from backend.pipeline.detectors.fusion_features). If < 0.85 widen it and re-
check; iterate. Record pct_golden_in_polygon. If the clip has NO clear secondary cluster to
exclude (courts overlap fully), say so ? a near-full-frame polygon is then a near no-op (an
honest null).

STEP 3 gate + score (numerically exact):
  from backend.pipeline.detectors.court_gate import gate_intervals_to_court
  gated = gate_intervals_to_court(baseline, points, polygon, FPS, FW, fh, min_in_court_frac=0.5)
  from backend.eval.metrics import score, match_intervals
  from backend.eval.tolerance_metrics import match_1to1
  b = score(baseline, GTS, 0.5); g = score(gated, GTS, 0.5)   # .precision/.recall/.f1
  base_tolf1 = match_1to1(baseline, GTS)[3]; gated_tolf1 = match_1to1(gated, GTS)[3]
  Of the DROPPED rallies (baseline minus gated), count how many were baseline FALSE POSITIVES: a
baseline pred is a FP if it is NOT in match_intervals(baseline, GTS, 0.5).matches as a matched
pred index.
  seg_ratio = n_pred / n_gt.

STEP 4 return the structured row. The win is REAL only if precision/F1 RISE with recall ~FLAT
(you cut FPs, not real rallies). If recall drops materially, or the clip barely over-fires, or
courts don't separate -> say so honestly (a null/negative result is a valid, valuable finding).
pct_golden_in_polygon < 0.85 means your polygon is unsafe ? fix it before reporting.
```

2. assistant (2026-06-19T16:29:46.421Z)

I'll verify this prominent-court-gate result adversarially. Let me start by ramping up on the repo state and the data/code involved.

3. user (2026-06-19T16:29:50.163Z)

```
Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end'
from the remote, but no such ref was fetched.
---BRANCH---
docs/court-floor-rally-end
---HEAD---
430de76 docs(rally): capture the shuttle-floor-contact rally-END signal (owner note)
```

4. user (2026-06-19T16:29:50.596Z)

```
output/Badminton_BXH_2_golden_features.json
output/human_lovo_manifest.json
---
keys ['name', 'fps', 'frame_width', 'candidates', 'gts']
n_gts 44
first [[0.413, 14.164], [22.666, 26.164], [36.665, 41.658]]
```

5. assistant (2026-06-19T16:29:53.842Z)

The branch is `docs/court-floor-rally-end`, not the one in git status. Let me note the pull failed (misconfigured upstream) and proceed read-only on local. Let me inspect the manifest entry and the relevant code modules.

6. user (2026-06-19T16:29:58.211Z)

```
manifest is list, len 15
ENTRY: {
  "name": "Badminton_BXH_2",
  "fps": 59.94,
  "frame_width": 1920.0,
  "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Badminton_BXH_2.rallies.csv",
  "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv"
}
```

7. user (2026-06-19T16:30:01.853Z)

```
-rw-r--r-- 1 avidu 197609 1549493 Jun 17 15:02 C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv
---HEAD---
Frame,Visibility,X,Y
0,1,822.7199935913086,531.8129539489746
1,1,823.1193351745605,533.6355400085449
2,1,824.5069313049316,536.7862129211426
3,0,-inf,-inf
```

8. user (2026-06-19T16:30:05.512Z)

```
windows_for OK
SERVED WindowingPreset(name='served', velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=2.0, max_window_duration=6.0, hard_cap=False, inactivity_end=1.5,
inactivity_rally_thresh=0.2, inactivity_min_density=0.3, blob_fallback=False, blob_factor=4.0)
parse_trajectory_csv True
point_in_polygon OK
gate_intervals_to_court sig: (intervals: 'List[Tuple[float, float]]', points: 'List[Any]',
polygon: 'Optional[Polygon]', fps: 'float', frame_width: 'float', frame_height: 'float',
min_in_court_frac: 'float' = 0.5, height_pad: 'float' = 0.0) -> 'List[Tuple[float, float]]'
score sig: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]], iou_threshold:
float = 0.5, match_result: Optional[backend.eval.metrics.MatchResult] = None) ->
backend.eval.metrics.Score
match_1to1 sig: (preds: 'List[Interval]', gts: 'List[Interval]', tau_start: 'float' = 2.0,
tau_end: 'float' = 1.5) -> 'Tuple[List[Match], float, float, float]'
```

9. assistant (2026-06-19T16:30:08.608Z)

All imports resolve. Now let me look at the key code to understand the exact semantics of `parse_trajectory_csv`, `point_in_polygon`, `gate_intervals_to_court`, and `match_intervals` before running.

10. user (2026-06-19T16:30:09.562Z)

```
"""Prominent-court shuttle gate ? multicourt G2 (see
``docs/PROMINENT_COURT_DETECTION.md``).

    In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
on a
    background / adjacent court becomes a false rally on the filmed match. This gate re-
defines the
    shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
a visible
    shuttle point whose normalised position is outside the prominent-court polygon is marked
NOT
    visible, so the velocity windower never opens a rally there.

    Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
    ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
visibility, never
    set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
    ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
    unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.

    The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
    ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
a point up
    to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
    admitted, so a high clear over the prominent court is not clipped.
    """
    from __future__ import annotations

    from dataclasses import replace
    from typing import Any, List, Optional, Sequence, Tuple

    from backend.pipeline.detectors.fusion_features import point_in_polygon

    Polygon = Sequence[Tuple[float, float]]


    def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
        """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?

        The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
        finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
        over the prominent court, admit it."""
        if point_in_polygon((nx, ny), polygon):
            return True
        return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)


    def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
                          fps: float, frame_width: float, frame_height: float,
                          height_pad: float = 0.0) -> float:
        """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.

        ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
        visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
        leave that judgement to the windower)."""
```

```
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return 1.0
        fs, fe = interval[0] * fps, interval[1] * fps
        n = inside = 0
        for p in points:
            if getattr(p, "visible", False) and fs <= p.frame <= fe:
                n += 1
                if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
                    inside += 1
        return 1.0 if n == 0 else inside / n


    def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
                                polygon: Optional[Polygon], fps: float,
                                frame_width: float, frame_height: float,
                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
                                ) -> List[Tuple[float, float]]:
        """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.

        Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
        occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
        window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
        point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
        so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
        intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return intervals
        return [iv for iv in intervals
                if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
                >= min_in_court_frac]


    def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
                             frame_width: float, frame_height: float,
                             height_pad: float = 0.0) -> List[Any]:
        """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
        polygon (expanded upward by ``height_pad``).

        No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
        are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
        its points are never mutated)."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
            return points
        out: List[Any] = []
        for p in points:
            if getattr(p, "visible", False):
                nx, ny = p.x / frame_width, p.y / frame_height
                if not admitted(nx, ny, polygon, height_pad):
                    p = replace(p, visible=False)
            out.append(p)
        return out

```

11. user (2026-06-19T16:30:10.219Z)

```
"""Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).

NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
math is unit-testable in a GPU-free dev container. The detection (person boxes,
shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
module only turns those signals into a handful of **scale/translation-invariant**
features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
memorise this court/camera and fail to generalise off a small labelled set).

Conventions:
- A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
- ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
  index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
- A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
- ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
  detection counts ? the player-count-only mode).
- ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).

Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
"""
from __future__ import annotations

from typing import Dict, List, Optional, Sequence, Tuple

BBox = Tuple[float, float, float, float]
Net = Tuple[str, float]


def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
    """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
    (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
    if polygon is None or len(polygon) < 3:
        return False
    x, y = point
    inside = False
    n = len(polygon)
    j = n - 1
    for i in range(n):
        xi, yi = polygon[i]
        xj, yj = polygon[j]
        # Does the horizontal ray at y cross edge (i, j)?
        if (yi > y) != (yj > y):
            x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
            if x < x_cross:
                inside = not inside
        j = i
    return inside


def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
    """Normalised foot point (bottom-centre) of a person box ? where the player stands,
    the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
    if frame_width <= 0 or frame_height <= 0:
```

```
55              return (0.0, 0.0)
56          x1, _y1, x2, y2 = box
57          return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60      def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61          if frame_width <= 0 or frame_height <= 0:
62              return (0.0, 0.0)
63          x1, y1, x2, y2 = box
64          return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67      def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69          """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70          True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71          point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
72          if court_polygon is None:
73              return True
74          if frame_width <= 0 or frame_height <= 0:
75              return False
76          return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
77
78
79      def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                          frame_height: float,
81                          court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82          """Per-frame count of in-court persons (one entry per sampled frame)."""
83          return [
84              sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
85              for _frame_idx, dets in sorted(per_frame.items())
86          ]
87
88
89      def _median(vals: Sequence[float]) -> float:
90          if not vals:
91              return 0.0
92          s = sorted(vals)
93          n = len(s)
94          return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97      def players_in_court(counts: Sequence[int]) -> float:
98          """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99          return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                            frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
```

```
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
```

```
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                    for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
```

12. assistant (2026-06-19T16:30:14.420Z)

Now let me understand `windows_for`, `parse_trajectory_csv`, and `match_intervals` semantics before running the measurement.

13. user (2026-06-19T16:30:16.355Z)

```
=== windows_for ===
def windows_for(video: Dict[str, Any], preset: WindowingPreset,
                min_crossings: int = 0) -> List[Interval]:
    """Parse a video's trajectory, window it under ``preset`` ? predicted rally intervals.

    ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
    ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than this
many
    times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only, derived
    from the trajectory + an auto-estimated net axis; it cleans up the over-split that bounded
    windowing (W1) leaves behind.
```

```
    NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate would
    drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate is
    strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no harm".
    """
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
    wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
    ivs = windowing.windows_to_intervals(wins)
    # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
    # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the prominent
court ? a
    # background/adjacent-court rally. Window-level (not point-level): a background rally's
central
    # tail keeps a few in-court points, enough to hold a window, so point-clearing alone is too
leaky
    # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
    poly = video.get("court_polygon")
    if poly and ivs:
        from backend.pipeline.detectors.court_gate import gate_intervals_to_court
        fw = float(video["frame_width"])
        fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
        ivs = gate_intervals_to_court(
            ivs, points, poly, float(video["fps"]), fw, fh,
            min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
            height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
    if min_crossings > 0 and ivs:
        from backend.pipeline.detectors.rally_gate import gate_windows
        gated = [(g.start, g.end)
                 for g in gate_windows(points, ivs, video["fps"], min_crossings=min_crossings)
                 if g.kept]
        # Gating emptied a non-empty video ? keep the ungated windows.
        ivs = gated if gated else ivs
    return ivs
```

14. user (2026-06-19T16:30:17.259Z)

```
=== match_intervals ===
def match_intervals(preds: List[Interval], gts: List[Interval],
                    iou_threshold: float = 0.5) -> MatchResult:
    """Greedy one-to-one matching of predictions to ground truth by descending IoU.

    Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
    are claimed highest-IoU first, and each prediction/GT can be used once. This
    keeps a single prediction from being credited for two ground-truth rallies
    (and vice versa).
    """
    candidates = []
    for pi, p in enumerate(preds):
        for gi, g in enumerate(gts):
            iou = interval_iou(p, g)
            # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold` would
            # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
            if iou > 0.0 and iou >= iou_threshold:
                candidates.append((iou, pi, gi))
    # Highest IoU first; ties broken by indices for determinism.
    candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

    used_pred, used_gt = set(), set()
    matches: List[Match] = []
    for iou, pi, gi in candidates:
        if pi in used_pred or gi in used_gt:
            continue
        used_pred.add(pi)
```

```
        used_gt.add(gi)
        matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

    unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
    unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
    return MatchResult(matches=matches,
                       unmatched_pred_indices=unmatched_pred,
                       unmatched_gt_indices=unmatched_gt)

=== MatchResult ===
@dataclass
class MatchResult:
    """Outcome of matching predictions to ground truth at a given IoU threshold."""
    matches: List[Match] = field(default_factory=list)
    unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
    unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives (misses)
```

15. assistant (2026-06-19T16:30:19.990Z)

I now understand all the semantics. Now let me reproduce the baseline measurement first, independently, and verify the reported baseline numbers (n_pred=78, P=0.346, R=0.614, F1=0.443).

16. user (2026-06-19T16:30:28.672Z)

```
FPS 59.94 FW 1920.0 n_gt 44
baseline n_pred 78
BASELINE precision=0.346154 recall=0.613636 f1=0.442623
base_tolf1=0.327869
n matches 27 unmatched_pred(FP) 51 unmatched_gt(miss) 17
```

17. assistant (2026-06-19T16:30:36.172Z)

Baseline reproduces EXACTLY: n_pred=78, P=0.346154, R=0.613636, F1=0.442623, base_tolf1=0.327869. And critically: 27 matches (TPs), **51 false positives**, 17 missed golden. So there ARE 51 baseline FPs available to potentially cut.

Now the key adversarial question: is there a genuinely separable adjacent court? Let me independently inspect the 2D shuttle distribution and the golden-rally medians, and also examine the FP medians vs TP medians.

18. user (2026-06-19T16:30:45.104Z)

```
total pts 43695 visible 29373
X  min 0.003 p05 0.074 p25 0.387 med 0.467 p75 0.469 p95 0.576 max 0.969
Y  min 0.000 p05 0.138 p25 0.344 med 0.347 p75 0.410 p95 0.598 max 0.940
Row=y(0 top..9 bot), Col=x(0 left..9 right); cell=count
      x0    x1    x2    x3    x4    x5    x6    x7    x8    x9
y0    101    18     8    89   246   351   150    34     0     0
y1    241    47    11   212   447   345    79   106     5     0
y2    678   302    86   486  1012   167    66    77    26     0
y3    863   472   110   945 13169   480    43    10    15     4
y4    335   224   222  1408  1984   513   117     2    15     6
y5     93    82   191   359   358   411   202     9    11     6
y6    125     0    13   402   473    42    21     6    33     8
y7     27     0     5    33     0     0    84     0    35    16
y8      0     0     0     0     0     0     0     0     0     0
y9      0     1     0     0     0     0     0     0     0     0
```

19. assistant (2026-06-19T16:30:49.946Z)

Key finding in the grid: there's a massive spike at cell (y3, x4) = 13169 points ? that's the static-tracker artifact parked near court-center (x~0.467, y~0.347), confirming the report's claim. The shuttle mass is concentrated in the left/center region (x0-x6, y0-y6). There's a sparse tail at x7-x9 and y7+ but very low counts.

Let me now check whether there's a separable secondary cluster, and compute the golden-rally medians to validate recall-safety. Let me also look at the column sums to see if x>0.68 (the right of the reported polygon) carries meaningful mass.

20. user (2026-06-19T16:31:00.377Z)

```
TOTAL visible 29373
x>0.68       (546, 0.018588499642528854)
x>0.70       (424, 0.014435025363429)
y>0.68       (275, 0.009362339563544752)
y<0.08       (721, 0.02454635209205733)
reported poly box x[0,0.68] y[0.08,0.68] (27936, 0.9510775201715862)
artifact cell x[0.4,0.5] y[0.3,0.4] (13169, 0.44833690804480303)
n golden with shuttle pts 44 of 44
golden median X: min 0.098 max 0.635
golden median Y: min 0.246 max 0.429
```

21. assistant (2026-06-19T16:31:07.716Z)

Now I have the key numbers. Let me verify:
- Reported polygon box x[0,0.68] y[0.08,0.68] contains 95.1% of all visible shuttle mass (so it excludes only ~4.9% ? essentially the sparse frame edges).
- x>0.68 carries only 1.9% of mass, y>0.68 only 0.9% ? confirming no spatially-separated secondary court cluster.
- Golden medians: X 0.098-0.635, Y 0.246-0.429 ? all well inside the reported polygon box (which goes to x=0.68, y=0.68). The report claimed X 0.098-0.638; I get max 0.635 (rounding, essentially identical).

Now let me directly run the gate with the reported polygon and verify pct_golden_in_polygon and that gating drops 0 (the no-op claim), then test the adversarial recall-safety: does tightening to "match golden" silently kill recall?

22. user (2026-06-19T16:31:21.771Z)

```
pct_golden_in_polygon = 44/44 = 1.0000
baseline n 78 gated n 78
GATED precision=0.346154 recall=0.613636 f1=0.442623
BASE  precision=0.346154 recall=0.613636 f1=0.442623
gated_tolf1=0.327869
n_dropped 0
n_dropped_were_fp 0
```

23. assistant (2026-06-19T16:31:28.169Z)

Everything reproduces exactly: pct_golden_in_polygon=1.0, gate drops 0/78, all metrics identical (no-op), n_dropped=0, n_dropped_were_fp=0.

Now the critical adversarial checks. The report claims "tightening the polygon to force drops immediately kills recall." Let me verify two things: (1) is there really NO separable adjacent court (the polygon isn't just drawn to match golden while hiding a real court), and (2) does a tighter polygon that excludes the sparse right/bottom regions actually cut real rallies, not just FPs? Let me test the claim that FPs are co-located with TPs by examining FP vs TP medians, and probe what a tighter gate would drop.

24. user (2026-06-19T16:31:41.031Z)

```
TP (matched) preds n=27: x[0.098,0.483] y[0.241,0.448]
FP (unmatched) preds n=51: x[0.060,0.497] y[0.201,0.620]
FP medians inside reported box: 51 / 51
TP medians inside reported box: 27 / 27
median x>0.55 : TP=0 FP=0
median x>0.60 : TP=0 FP=0
median x>0.65 : TP=0 FP=0
```

25. assistant (2026-06-19T16:31:47.571Z)

This is decisive confirmation of the report's core claim. Both TP and FP medians live entirely inside the prominent court:
- TP medians: x[0.098, 0.483], y[0.241, 0.448]
- FP medians: x[0.060, 0.497], y[0.201, 0.620]

The FP x-range (0.060-0.497) is essentially identical to and overlaps the TP x-range (0.098-0.483). All 51 FP medians AND all 27 TP medians are inside the reported box. No FP median sits past x=0.55. So there is genuinely no spatial separation between false positives and true positives ? a court polygon cannot discriminate them. The FPs are not on an adjacent court; they're co-located with the real rallies in the prominent court (the static-tracker artifact + over-splitting).

Now let me adversarially test the "tightening kills recall" claim directly: apply progressively tighter polygons and confirm any drop hits real rallies (recall loss), not FPs.

26. user (2026-06-19T16:31:59.535Z)

```
reported box x.68/y.68  -> gated n=78 dropped=0 (TP=0 FP=0) P=0.346 R=0.614 F1=0.443
tighter x<=0.55        -> gated n=78 dropped=0 (TP=0 FP=0) P=0.346 R=0.614 F1=0.443
tighter x<=0.50        -> gated n=78 dropped=0 (TP=0 FP=0) P=0.346 R=0.614 F1=0.443
tighter x<=0.48        -> gated n=75 dropped=3 (TP=2 FP=1) P=0.333 R=0.568 F1=0.420
tighter y<=0.50        -> gated n=76 dropped=2 (TP=1 FP=1) P=0.342 R=0.591 F1=0.433
tighter y>=0.20        -> gated n=77 dropped=1 (TP=1 FP=0) P=0.338 R=0.591 F1=0.430
```

27. assistant (2026-06-19T16:32:06.296Z)

This fully confirms the report's "tightening kills recall" claim. The moment a polygon is tight enough to drop ANYTHING (x<=0.48), it drops 2 TPs vs 1 FP ? recall falls from 0.614 to 0.568 and F1 drops from 0.443 to 0.420. Every tightening that drops anything hits real rallies. There is no polygon that cuts FPs without cutting TPs. The gate genuinely cannot win here.

Let me do one final adversarial sanity check: confirm the static-tracker artifact claim (~42-44% of points parked at one spot) is real and not fabricated, and double-check the golden-median y-range and reported "FP median x 0.060-0.497 == TP x 0.098-0.484" claim matches mine (I got TP x max 0.483, report said 0.484 ? let me verify that's just rounding from a single point).

28. user (2026-06-19T16:32:13.333Z)

```
Top-5 most-frequent rounded shuttle positions (frac of visible=29373):
  (0.47,0.35) count=10445 frac=0.356
  (0.47,0.34) count=1526 frac=0.052
  (0.51,0.60) count=198 frac=0.007
  (0.41,0.61) count=142 frac=0.005
  (0.34,0.62) count=94 frac=0.003
Most-frequent EXACT pixel coord: (896.25,371.25) -> norm (0.467,0.344) count=831 frac=0.028
```

29. assistant (2026-06-19T16:32:26.092Z)

Confirmed: a static-tracker artifact parked at norm (0.467, 0.347) ? the single rounded cell (0.47,0.35) holds 35.6% of all visible points, and (0.47,0.34)+(0.47,0.35) together ~41% ? matching the report's "~42% parked at court-center 0.467,0.347" claim closely.

Now I have verified every component of the reported row. Let me summarize the verdict:

Adversarial verification results:

1. **Polygon recall-safe?** YES. pct_golden_in_polygon = 44/44 = 1.0 (reproduced exactly). All 44 golden medians (x 0.098-0.635, y 0.246-0.429) sit well inside the box.

2. **Does it exclude a genuine adjacent court?** NO adjacent court exists. x>0.68 carries 1.9% of mass, y>0.68 carries 0.9% ? no separable secondary cluster. 95.1% of shuttle mass is in the box; the excluded ~4.9% is sparse frame-edge noise, not a court. The polygon is honestly a near-

full-court no-op, NOT drawn to hide a real court.

3. **Precision/F1 gain real? Dropped rallies genuine FPs?** There IS no gain (correctly reported as null). Gate drops 0/78; all metrics identical. n_dropped=0, n_dropped_were_fp=0. The 51 baseline FPs are spatially co-located with the 27 TPs (FP median x[0.060,0.497] vs TP x[0.098,0.483]; ALL 51 FP medians inside the box) ? a court gate cannot separate them.

4. **Recall silently lost?** NO. delta_recall=0 is correct (gate is a no-op). I further confirmed that any tighter polygon that drops anything hits real rallies first (x<=0.48 drops 2 TP + 1 FP, recall 0.614?0.568) ? corroborating the "tightening kills recall" read rather than contradicting it.

5. **Any number inflated?** NO. Every reported number reproduces exactly. The only trivial discrepancy: report says TP x-max 0.484 vs my 0.483 (rounding) ? immaterial.

The reported row is fully accurate and the read is honest. Returning the verified row (numbers confirmed; I refine the golden-X-max to my measured 0.635 and note it in the polygon summary, and tighten the read slightly with my corroborating tighten-test evidence).

30. user (2026-06-19T16:32:37.142Z)

Structured output provided successfully

31. assistant (2026-06-19T16:32:50.328Z)

Verification complete. The reported row is fully accurate and the read is honest ? every number reproduced exactly, and the adversarial checks corroborated rather than contradicted the null claim.

Summary of what I independently verified:

- **Baseline reproduced exactly**: n_pred=78, P=0.346154, R=0.613636, F1=0.442623, base_tolf1=0.327869. Match breakdown: 27 TPs, 51 FPs, 17 missed golden.
- **Polygon is genuinely recall-safe AND not gerrymandered**: pct_golden_in_polygon=44/44=1.0; the box holds 95.1% of shuttle mass; x>0.68 and y>0.68 carry only 1.9% / 0.9% ? there is no separable adjacent court to exclude. The polygon is an honest near-full-court no-op, not drawn to hide a real court.
- **No real gain, correctly reported as null**: gate drops 0/78, all metrics identical; n_dropped=0, n_dropped_were_fp=0.
- **FPs are co-located with TPs, not on an adjacent court**: FP median x[0.060,0.497] overlaps TP x[0.098,0.483]; all 51 FP medians sit inside the box. The ~41% static-tracker artifact at norm (0.467,0.347) is real (single rounded cell = 35.6% of points).
- **Recall was NOT silently lost** (delta_recall=0 is correct), and I confirmed adversarially that the only way to make the gate drop anything is a polygon tight enough to hit real rallies first (x<=0.48 ? drops 2 TP vs 1 FP, recall 0.614?0.568) ? corroborating the "tightening kills recall" read.

The only trivial discrepancy: the report's golden-X-max of 0.638 vs my measured 0.635, and TP-x-max 0.484 vs my 0.483 ? both rounding-level and immaterial. No inflation found.

Relevant files:
- `C:/Users/avidu/Projects/badminton-highlight-indexer/output/Badminton_BXH_2_golden_features.json` (golden labels)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json` (manifest entry, fps=59.94)
- `C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv` (trajectory)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/court_gate.py` (gate under test)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/rally_seg_eval.py` (`windows_for`)

One process note: `git pull --ff-only` failed because the current branch `docs/court-floor-rally-end` has a misconfigured upstream (`refs/heads/docs/court-floor-rally-end` not fetched); I proceeded read-only on the local checkout, which is appropriate for a pure verification task

that wrote only a throwaate pickle to `output/`.